

Bioinformatics workshop setup — a step-by-step guide for absolute beginners

Before you start	2
Which steps are mine?	2
How to read these instructions	2
The three terminals, and which to use	3
Official downloads	3
Part 1 — Get a Linux terminal	5
Module 1 — Install Ubuntu on Windows (WSL)	5
Module 2 — Get your Mac ready	5
Part 2 — Install conda and build the workshop environment	7
Module 3 — Miniforge and the <code>protein_modeling</code> environment	7
Part 3 — Git, GitHub, and VS Code	11
Module 4 — Install and set up Git	11
Module 5 — Create a GitHub account	11
Module 6 — Install VS Code and publish a practice repository	11
Part 4 — Add the metagenomics toolbox	13
Module 7 — The <code>meta_env</code> environment	13
Part 5 — Finishing and fixing	16
Everything you should now have	16
If something fails	16
Sources	17

This guide takes a laptop with nothing installed and leaves you with a working bioinformatics setup: a Linux terminal, the conda package manager, the workshop's `protein_modeling` software environment, the Git and VS Code tools used in the later sessions, and — if your session includes metagenomics — a second environment holding the assembly and annotation tools. It assumes you have never opened a terminal before and explains each step as it comes.

Work through it in order, one line at a time. Every module ends with a **checkpoint** — a short thing you can show an instructor to prove that part worked. If a command fails, stop at that line and ask; carrying on usually makes the problem harder to find.

Before you start

You need	Why
A Windows or Mac laptop you can install software on	WSL and conda both install software
Permission to install	On an institution-managed machine, ask IT first
A stable internet connection	Several gigabytes are downloaded
About 10 GB of free disk space	Ubuntu, conda, and the workshop tools
A further 20 GB, for Module 7 only	The metagenomics tools, their database, and assembly output
About 90 minutes	Plus one Windows restart, on Windows. Module 7 adds about 40 minutes
Your laptop's own password	Windows or Mac will ask during installation

When you finish you will be able to open a terminal, type `conda activate protein_modeling`, and have Python, Biopython, pandas, PyMOL, and the bioinformatics tools ready to use. Module 7 adds a second toolbox, `meta_env`, for the metagenomics session.

Which steps are mine?

Find your computer in this table and follow only that row. Nothing is skipped by accident — the modules you leave out genuinely do not apply to your machine.

Your computer	Do these modules
Windows 10 or 11	1 → 3 (Ubuntu/WSL instructions) → 4 → 5 → 6 → 7
Mac (Apple silicon or Intel)	2 → 3 (Mac instructions) → 4 → 5 → 6 → 7

Modules 5 and 6 include publishing to GitHub, which is optional; the local parts are not. **Module 7 is only for sessions that cover metagenomics** — if yours does not, stop after Module 6.

Windows users: the surprise worth knowing now. You will install Ubuntu inside Windows and do the real work there, not in PowerShell. The workshop's tools come from bioconda, which only builds software for Linux and macOS. Installing Ubuntu through WSL is the supported way to run them on a Windows laptop, and Module 1 does exactly that.

How to read these instructions

You see	It means
A box with a blue bar on the left	Something you type, then press Enter
A box with a grey bar on the left	Roughly what your screen shows back
<code><like_this></code>	A placeholder — replace it, angle brackets and all
Checkpoint	Proof that the module worked. Do not move on without it
[1], [2] ...	A link to the official documentation, listed under Sources

You see	It means
Video alternative	A YouTube walkthrough of the same steps, if you would rather watch first. Where a video and this guide differ, follow the guide, and download from <i>Official downloads</i> , not from a video's description

- **One line at a time.** Installers ask questions in the middle of a run; pasting a whole block answers them for you, at random.
- **Passwords stay invisible.** When Ubuntu asks for a password, nothing appears as you type — no dots, no stars. That is normal. Type it and press Enter.
- **sudo means “as administrator”.** It will ask for your Ubuntu password the first time in each session.
- **Downloads can be slow.** A command that seems frozen is usually still downloading. Leave it be.

Three terms that are used throughout: `~` is your home folder; `cd` moves you into a folder; `pwd` prints which folder you are in.

The three terminals, and which to use

A terminal is just a window where you type a command and press Enter. You will meet three of them, and they are not interchangeable.

Terminal	How to open it	Its prompt	What it is for here
Windows PowerShell	Start menu → type <i>PowerShell</i>	PS C:\>	Module 1 only: installing and checking WSL
Ubuntu (WSL)	Start menu → Ubuntu (after Module 1)	you@laptop:~\$	Everything else, on Windows
Mac Terminal	Cmd+Space → type <i>Terminal</i>	you@MacBook ~ %	Everything, on a Mac

Each step below names the terminal it belongs in. When a step says Ubuntu, PowerShell will not do.

Official downloads

In most cases you will not need to click these: the commands in the modules download the right file for you. They are listed so you can check that what you have came from the official source. Never install conda, Git, or VS Code from a third-party download site.

- WSL + Ubuntu — installed by `wsl --install`; Microsoft's guide: <https://learn.microsoft.com/en-us/windows/wsl/install>
- Ubuntu Miniforge, x86_64 (`.sh`): https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh
- Ubuntu Miniforge, aarch64 (`.sh`): <https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-aarch64.sh>
- Mac Miniforge, Apple silicon arm64 (`.pkg`): <https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-MacOSX-arm64.pkg>
- Mac Miniforge, Intel x86_64 (`.pkg`): https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-MacOSX-x86_64.pkg
- Windows Miniforge (`.exe`), optional and **not** used to build the workshop environment — only for the Windows-native PyMOL fallback in Module 3: https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Windows-x86_64.exe
- PyMOL, open-source — the conda-forge package, downloaded by conda in Module 3: <https://anaconda.org/conda-forge/pymol-open-source>. This one package is both the viewer window and the command-line program. The installer and licence on pymol.org are Schrödinger's separate commercial build; the workshop does not need them.
- Git — Git for Windows, and the macOS options: <https://git-scm.com/downloads>
- VS Code for Windows or Mac: <https://code.visualstudio.com/download>
- GitHub account: <https://github.com/signup>

- The rest of the workshop software (Biopython, pandas, the bioinformatics tools) is also downloaded by conda in Module 3. There is no separate installer.

Part 1 — Get a Linux terminal

Module 1 — Install Ubuntu on Windows (WSL)

Windows only · about 30 minutes, including one restart · Mac users skip to Module 2.

Video alternative: *Install WSL on Windows 11 Fast: Ubuntu & Debian Step-by-Step Guide* (MadTc Tech, 18 min): <https://www.youtube.com/watch?v=FI8tqwv4KEM>. It uses the same `wsl --install` command as Step 2; the Debian part is not needed.

WSL is Microsoft's supported way of running real Ubuntu inside Windows. Windows 11, or Windows 10 build 19041 or newer, supports the one-command installation [1]. If you also want Linux programs to open their own windows, that needs Windows 11 or Windows 10 build 19044+ with WSL 2 [2]. You can check your build under **Settings → System → About**.

Step 1. Open PowerShell as an administrator. Click Start, type *PowerShell*, and choose **Run as administrator**. Confirm the Windows prompt that appears.

Step 2. Install WSL and Ubuntu with one command.

```
wsl --install
```

This downloads both WSL and Ubuntu, which takes several minutes. When it asks you to restart Windows, do it.

If it only prints a page of help text, WSL is already present. Run `wsl --list --online` to see the available distributions, then `wsl --install -d Ubuntu` [1].

Step 3. Start Ubuntu and create your Linux account. After the restart, open **Ubuntu** from the Start menu. The first launch takes a minute, then asks for a username and password of your own choosing — these are new, separate from your Windows login. **Nothing appears on screen while you type the password.** That is deliberate; type it and press Enter.

Step 4. Confirm Windows can see Ubuntu. Open a *PowerShell* window (not Ubuntu) and run:

```
wsl --list --verbose
```

NAME	STATE	VERSION
* Ubuntu	Running	2

The `2` matters: it means WSL 2. If it shows `1`, ask the instructor before continuing.

Step 5. Set Ubuntu up for work. Back in the **Ubuntu** window:

```
uname -m
sudo apt update
sudo apt install -y curl ca-certificates git
mkdir -p ~/workshop
cd ~/workshop
pwd
```

```
x86_64
/home/you/workshop
```

Write down what `uname -m` printed — `x86_64` or `aarch64`. It decides which Miniforge installer you need in Module 3.

`sudo` will ask for the Ubuntu password you just created. The last line must start with `/home/`; keep your Linux projects there rather than under `/mnt/c/`, where file access is much slower [3].

Checkpoint: PowerShell lists Ubuntu as version `2`, and in Ubuntu `pwd` prints a path beginning with `/home/`.

Module 2 — Get your Mac ready

Mac only · about 5 minutes.

Step 1. Open Terminal. Press `Cmd+Space` , type *Terminal*, press Enter.

Step 2. Check your macOS version and chip, and make your folders.

```
sw_vers
uname -m
mkdir -p ~/workshop/data ~/workshop/results
cd ~/workshop
pwd
```

```
ProductName:    macOS
ProductVersion: 14.5
arm64
/Users/you/workshop
```

Current Miniforge needs macOS 11 or newer [4]. Write down what `uname -m` printed: `arm64` means Apple silicon (M1 and later), `x86_64` means an Intel Mac. That choice decides which installer you download in Module 3 — the wrong one will not run.

Keep original data in `~/workshop/data` and everything you produce in `~/workshop/results` , so you can always tell them apart. Do not replace or remove the Python that comes with macOS; conda will install its own. Apple's Command Line Tools are not needed for this workshop, but if something later asks for them, run `xcode-select --install` and follow Apple's prompt.

Checkpoint: you know whether your Mac is `arm64` or `x86_64` , and `cd ~/workshop` works.

Part 2 — Install conda and build the workshop environment

Module 3 — Miniforge and the `protein_modeling` environment

Everyone · about 30 minutes, mostly waiting for downloads.

Miniforge is a small installer that gives you **conda**, the program that installs scientific software into self-contained **environments** [4, 5]. An environment is a named box of software you switch on when you need it, which is how you keep one project's tools from breaking another's.

Follow **one** of the two installation sections below, then do *Build the workshop environment*, which everyone does. If conda is already installed and working on this machine, ask the instructor before adding a second copy.

Windows users: which section is mine?

Yours is **Install Miniforge on Ubuntu/WSL**, typed in the Ubuntu window from Module 1. The native Windows Miniforge `.exe` cannot build this workshop's environment, because the bioinformatics packages are not published for Windows; install it later only if you want Windows-native conda practice for something else.

Install Miniforge on Ubuntu/WSL

Video alternative: *Installing Conda Using the Miniforge Installer* (Linux Admin for Bioinformaticians, 5 min): <https://www.youtube.com/watch?v=8AeITlbD0wc>. It is filmed on Linux Mint, and the steps are the same in Ubuntu/WSL. Its last step, turning off the automatic `(base)` environment, is optional.

In the **Ubuntu** terminal — never PowerShell — download the installer that matches what `uname -m` printed. The `$(uname -m)` in the command fills that in for you:

```
cd ~
base=https://github.com/conda-forge/miniforge/releases/latest/download
curl -fL -o Miniforge3.sh "$base/Miniforge3-Linux-$(uname -m).sh"
ls -lh Miniforge3.sh
```

```
-rw-r--r-- 1 you you 78M Oct  7 09:12 Miniforge3.sh
```

That size should be in the tens of megabytes. A file of a few kilobytes means the download failed and saved an error page instead — delete it and try again rather than running it. Then start the installer:

```
bash Miniforge3.sh
```

It asks three things: press Enter and then `q` to get through the licence, type `yes` to accept it, press Enter to accept the default location in your home folder, and answer `yes` when it offers to initialise your shell. **Close the Ubuntu window and open a new one** — conda only appears in terminals started after installation.

```
conda --version
```

```
conda 24.9.2
```

If conda: command not found, the shell initialisation was skipped. Run `~/miniforge3/bin/conda init bash`, then close and reopen Ubuntu [4].

Install Miniforge on Mac

Download the one `.pkg` from *Official downloads* that matches your `uname -m` result, then open it and follow the macOS installer. Accept shell initialisation if it is offered. **Close Terminal and open a new one**, then check:

```
conda --version
```

Use Miniforge's official installer rather than a Homebrew repackaging of it [4].

Video alternative: *Install Miniforge on Windows & Mac — The Free conda & mamba Installer (2026)* (ErrorFixer, 6 min). This link starts at the macOS `.pkg` part, at 3:03: <https://www.youtube.com/watch?v=tODVq4JvbU&t=183s>

Build the workshop environment — everyone

The whole workshop runs in **one** environment, `protein_modeling`, holding every tool the later sessions need, so you never have to switch environments mid-exercise. Build it in your newly reopened Ubuntu or Mac terminal.

If your instructor has given you the `Ologist_Workshop` project folder, `cd` into it and run the setup script, which does everything in one step:

```
bash setup_protein_modeling_conda_envs.sh
```

If you do not have the project folder, run the same command the script runs. Type `y` when conda asks you to confirm the package list. This is a large install — several minutes of downloading is normal, and the screen will fill with package names:

```
conda create -n protein_modeling \
  -c conda-forge -c bioconda python=3.11 \
  biopython pandas pymol-open-source \
  fastqc seqkit bwa samtools bcftools blast \
  vina meeko rdkit
```

The backslash at the end of a line means “this command continues below”. Type or paste all five lines, then press Enter once.

Either way, switch the environment on and test it:

```
conda activate protein_modeling
python --version
python -c "import Bio, pandas, pymol; print('biopython', Bio.__version__, 'pandas',
      pandas.__version__)"
conda info --envs
```

```
(protein_modeling) you@laptop:~$
Python 3.11.9
biopython 1.85 pandas 2.2.3

# conda environments:
#
base                /home/you/miniforge3
protein_modeling    * /home/you/miniforge3/envs/protein_modeling
```

Four things confirm success: `(protein_modeling)` at the front of your prompt, Python reporting `3.11`, two package versions printed without an error, and the `*` sitting beside `protein_modeling`. Your version numbers will differ from the ones above.

`conda deactivate` switches the environment off again. You will type `conda activate protein_modeling` at the start of every future session — installing happened once, activating happens every time.

Two habits to keep: never run `conda` with `sudo`, and never install workshop packages into `(base)`. If the environment already exists, activate it instead of creating it again; the setup script checks for you and skips it.

Background video, if environments still feel abstract: *Demystifying Conda (Anaconda, Miniconda and Bioconda) and Virtual Environments* (Bioinformagician, 36 min): <https://www.youtube.com/watch?v=2bXGm0ZnJ38>. It explains what an environment is and why bioinformaticians use them, so it is for understanding rather than a set of steps to copy.

Open PyMOL — check the viewer window

PyMOL, the program that draws protein structures, came in with the environment: the `pymol-open-source` package [29] is the whole viewer, so there is nothing more to download. A version number proves it installed; only opening the window proves you can use it:

```
conda activate protein_modeling
pymol --version
pymol
```

A window should open with a black 3-D canvas and a `PyMOL>` command line. Leave the terminal you started it from open — closing it closes PyMOL too. Quit with **File → Quit**.

On a Mac the window simply opens. On Windows it is drawn by WSLg, the part of WSL that shows Linux windows on your Windows desktop, which needs Windows 11 or Windows 10 build 19044+ [2]. If no window appears, run `wsl --update` and then `wsl --shutdown` in **PowerShell**, reopen Ubuntu, and try `pymol` again.

Windows too old for WSLg? Install a Windows-native PyMOL for viewing only. Install the Windows Miniforge `.exe` from *Official downloads*, open **Miniforge Prompt** from the Start menu, and run `conda create -n pymol_gui -c conda-forge pymol-open-source`, then `conda activate pymol_gui`, then `pymol`. That copy only views structures — every other command in the workshop still runs in Ubuntu.

Video alternative: *How to Install Open-Source PyMOL in a Conda Environment on Windows* (Chandra Veda Labs, 2 min): <https://www.youtube.com/watch?v=6YX028lvXuM>. It uses Anaconda or Miniconda; the commands are the same in Miniforge Prompt.

Also useful: your system's own package manager

Conda is not the only way to install software, and it is not always the right one. Conda installs research tools into an *environment*, where they exist only while that environment is active. Your operating system has its own package manager for general command-line utilities, which install into the machine and work in every terminal regardless of conda. On Ubuntu and WSL that is `apt`; on a Mac it is **Homebrew**.

Two small tools make the rest of the workshop easier, and they are a good first exercise. `tree` prints a folder and its contents as an indented tree, so you can see at a glance what a command produced. `btm` shows CPU, memory and disk live, which is how you tell a long-running job from a stuck one.

On Ubuntu or WSL, in the Ubuntu terminal. `sudo` means this installs for the whole system, so it asks for your Ubuntu password. Always refresh the package list first — skipping `apt update` on a fresh system is the usual reason a package “cannot be found” [13]:

```
sudo apt update
sudo apt install -y tree btm
tree --version
```

The last line should answer with a version number. If `btm` is not offered on an older Ubuntu, install `htop` instead; it does the same job.

On a Mac, check for Homebrew first:

```
brew --version
```

If that answers `command not found`, install Homebrew using the command published on <https://brew.sh> [14] — copy it from that page rather than from this PDF, so you can see it comes from the project itself. It will ask for your Mac password and take a few minutes. Then:

```
brew install tree btm
tree --version
```

Try it straight away, on the folder you made earlier:

```
tree -L 2 ~/workshop
```

sudo belongs to apt , never to conda. apt installs system-wide and genuinely needs permission. Conda installs into your own home folder and does not — sudo conda creates files your normal account can no longer edit, and it is one of the hardest problems to undo. Homebrew likewise is used without sudo .

If you are on a managed laptop where sudo is refused, you are not stuck: conda install -c conda-forge tree puts the same tool in your environment without needing administrator rights.

Checkpoint: show a partner your prompt with (protein_modeling) in it, the package-version line, the PyMOL window, and tree -L 2 ~/workshop drawing your folders.

Part 3 — Git, GitHub, and VS Code

Module 4 — Install and set up Git

Everyone · about 10 minutes.

Video alternative: A brief introduction to Git for beginners (GitHub, 9 min): <https://www.youtube.com/watch?v=r8jQ9hVA2qs>. It covers installing Git, setting your name and email, and what a commit is.

Git keeps a history of changes to your files, on your own machine. GitHub is a website that can host a copy of that history; it is optional, and covered in Modules 5 and 6. Install Git **in the same system your project lives in** [6, 7]:

- **Ubuntu/WSL** (this includes Windows users): run `git --version` in Ubuntu. Module 1 already installed it. If it is missing, run `sudo apt update` then `sudo apt install -y git`.
- **Mac:** run `git --version` in Terminal. If macOS asks for developer tools, run `xcode-select --install`, finish Apple's prompt, reopen Terminal, and check again.
- **A Windows-native project** (not inside WSL): install Git for Windows from *Official downloads* and use the **Git Bash** window it provides. If you work only in Ubuntu/WSL, you do not need this.

Now tell Git who you are. Replace both values with **your own** name and a verified GitHub email address — or the no-reply address GitHub can give you, if you would rather keep your real one private. These appear in the history of anything you publish, and they are not login credentials [7]:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global --get user.name
git config --global --get user.email
git --version
```

```
Your Name
you@example.com
git version 2.43.0
```

Do not leave the example name and address in place. `--global` means “for this operating system account”, so Windows Git and WSL Git keep separate settings and each needs configuring once.

Checkpoint: `git --version` works, and both `--get` commands print the values you intended.

Module 5 — Create a GitHub account

Optional but recommended · about 10 minutes.

Video alternative: How to Create a GitHub Account (For Beginners!) (The Common Coder, 8 min): <https://www.youtube.com/watch?v=RnWDKWmaQ8s>. It was recorded in 2024, so GitHub's sign-up screens may look slightly different now; the steps are the same.

Go to <https://github.com/signup>, create an account, verify the email address, and follow GitHub's security prompts [8]. Turn on two-factor authentication if it is not already required.

In **Settings** → **Emails**, decide whether to keep your address private, and use whichever address you set in Module 4 so your commits are attributed correctly. Keep passwords, recovery codes, and access tokens out of files and out of Git history. You do not need an account to practise Git locally, only to publish.

Checkpoint: you can sign in at `github.com` in a browser. Never show anyone your password, including the instructor.

Module 6 — Install VS Code and publish a practice repository

About 25 minutes.

Video alternatives:

- Getting around VS Code — *Learn Visual Studio Code in 15 minutes: 2026 Official Beginner Tutorial* (Visual Studio Code, 15 min; the Chat and MCP parts are not needed): https://www.youtube.com/watch?v=f8_uF_IDV50
- Windows: opening your Ubuntu folder in VS Code — *How to Connect to WSL in VS Code* (Ferds the NetDev, 6 min): <https://www.youtube.com/watch?v=q74CP4fB7cY>
- Steps 1–3 below — *How to use Git and GitHub in VS Code* (GitHub, 9 min; the AI-tools part is not needed): <https://www.youtube.com/watch?v=NFjz1AGKA4c>

VS Code is the editor used for the rest of the workshop. Install it from *Official downloads* [9].

On Windows, install VS Code on Windows itself, not inside WSL [10]. Then open it and install Microsoft's **WSL** extension.

From the Ubuntu terminal, `cd ~/workshop` and run `code .`; VS Code opens and shows **WSL: Ubuntu** in the bottom-left corner. That corner label is how you know you are editing Linux files rather than Windows ones.

Mac users, and Windows users not using WSL, open a folder with **File → Open Folder** instead.

Check that VS Code can see Git: open **Terminal → New Terminal** inside VS Code and run `git --version`. If it fails, restart VS Code — it only notices newly installed programs on startup.

Practise in a **new, empty** folder, never in a real research project. On Ubuntu or Mac:

```
mkdir -p ~/workshop/git-practice
cd ~/workshop/git-practice
code .
```

Windows users not using WSL can create the folder in File Explorer and open it with **File → Open Folder**. Only trust a folder you made yourself or know to be safe.

1. **Make a file and a repository.** In VS Code create `README.md`, type `# Git practice`, and save it. Open **Source Control** (the branch icon, or `Ctrl+Shift+G` / `Cmd+Shift+G` on Mac) and choose **Initialize Repository**. This creates a *local* history only — nothing is uploaded [11].
2. **Make your first commit.** Review the changed file, stage **only** `README.md` with its `+` button, type a message such as `Add workshop README`, and select **Commit**. Then confirm it in VS Code's terminal:

```
git status
git log -1 --oneline
```

```
On branch main
nothing to commit, working tree clean
alb2c3d Add workshop README
```

1. **Optional — publish it.** In Source Control, choose **Publish Branch / Publish to GitHub**, and pick a **private** repository for this exercise. Follow the browser sign-in, and read the list of files and the visibility setting before confirming [12]. Open the repository URL afterwards and check that `README.md` is there. From then on, commits stay local until you **Push**. If a push fails, stop and ask — do not force-push and do not sync repeatedly.

Never publish patient data, unpublished research data, passwords, `.env` files, access tokens, or large raw sequencing files. Check your institution's data-sharing rules before putting any real data on GitHub.

Checkpoint: show your local commit. If you published, show the **private** repository and its `README.md` — without showing credentials on screen.

Part 4 — Add the metagenomics toolbox

Module 7 — The `meta_env` environment

Only if your session includes metagenomics · about 40 minutes, plus a long database download.

The metagenomics session uses a different set of programs: quality-control tools, two assemblers, an evaluation tool, a graph viewer, and the annotation and screening tools. They go into a **second** environment, `meta_env`, rather than into `protein_modeling`. Two reasons: the assemblers bring large dependency sets of their own that can force conda to downgrade packages the protein session needs, and a broken toolbox is much easier to delete and rebuild when it is not also the toolbox everything else depends on.

You will end up with two environments and activate whichever the session needs. Only one is ever active at a time.

Behind `meta_env` sit two helper environments: `meta_annot_env` for Bakta and AMRFinderPlus, and `meta_abricate_env` for ABRicate. Those three tools need versions of shared libraries, such as BLAST and zlib, that clash with the assemblers' and with each other's, so no single environment can hold them all. The setup script links their commands into `meta_env`, so you never activate the helpers yourself.

Is my laptop up to this?

Your machine	What to expect
Linux x86_64, including Windows + WSL Ubuntu	Everything in this module works. This is the configuration the guide is written for
Linux or WSL on ARM64	Most packages are published for ARM; check each one installs rather than assuming
Mac, Intel	The core tools are available
Mac, Apple silicon	Most current packages have a <code>arm64</code> builds. OPERA-MS is the likely exception — see Step 5
Windows, PowerShell only	Will not work. These are Linux and macOS packages; use the Ubuntu window from Module 1

Assembly is the demanding step: expect it to want several gigabytes of RAM and a similar amount of disk, and expect the annotation database to be the biggest download of the workshop. If your laptop cannot manage it, that is a normal outcome and not a failure — ask about the workshop server before spending the session fighting it.

Do not install any of this into `(base)`. `base` is conda's own environment; filling it with research tools is how people end up reinstalling conda. Everything here goes into the named environments created in Steps 2 and 3.

If you have the project folder, one script replaces Steps 1–4. `cd` into `Ologist_Workshop` and run:

```
bash setup_metagenomics_conda_envs.sh
```

It creates `meta_env` and its two helpers, installs every tool, checks that each one prints a version from inside `meta_env`, and downloads the databases into `~/workshop/databases`. Add `--no-db` to install only the tools and fetch the databases later. The Bakta database is a 1.3 GB download from Zenodo, which can be slow and can break off partway; if it does, run the script again and it resumes where it stopped. The summary at the end prints the path to pass to `bakta` as `--db`; write it down. If the script reports that the annotation tools could not be installed, the tools up to the assembly graph are still there; use the workshop server for annotation and screening. Then continue at Step 5. Without the project folder, follow Steps 1–4 by hand.

Video alternative for Steps 1–3: *How to install bioinformatics software with mamba (faster version of conda)* (BigDataBiology-Lab, 12 min): <https://www.youtube.com/watch?v=GEZn3fsrSdE>. It shows the same mamba-and-bioconda pattern with other tools; type the package names from this guide, not the video's.

Step 1. Check you have `mamba`. Mamba resolves these larger package sets faster than conda alone. Recent Miniforge installs include it [4, 16]:

```
conda --version
mamba --version
```

If the second command answers `command not found`, install it once, into `base` — this is the one thing that belongs there, because it is a package manager rather than a research tool:

```
conda install -n base -c conda-forge mamba
```

Step 2. Create the environment. The two `-c` flags name the channels, and `--strict-channel-priority` tells conda to prefer the first one it is offered rather than mixing builds from both, which is what bioconda expects [15]:

```
mamba create -n meta_env -c conda-forge -c bioconda \
  --strict-channel-priority python=3.10
conda activate meta_env
```

Your prompt should now start with `(meta_env)`.

Step 3. Install the pipeline tools. This is in two parts on purpose. The first gives you everything up to the assembly graph; if the second has trouble, you can still do the whole morning's work:

```
mamba install -c conda-forge -c bioconda --strict-channel-priority \
  fastqc nanostat fastp fastplong spades flye quast bandage
```

The annotation and screening tools go into the two helper environments:

```
mamba create -n meta_annot_env -c conda-forge -c bioconda --strict-channel-priority \
  python=3.10 "bakta>=1.12" "ncbi-amrfinderplus>=4.2"
mamba create -n meta_abricate_env -c conda-forge -c bioconda --strict-channel-priority \
  abricate
```

Keep the version floors. Without them the solver can settle on an old Bakta whose AMRFinderPlus can no longer update its database. Several minutes of downloading per command is normal. Type `y` when you are asked to confirm the package list.

Installed by hand, the helpers are not linked into `meta_env`, so switch environments for these three tools: `conda activate meta_annot_env` before `bakta` or `amrfinder`, and `conda activate meta_abricate_env` before `abricate`.

Step 4. Check every tool answers. Run the lot. Each line should print a version, not an error [17–27]:

```
fastqc --version
NanoStat --version
fastp --version
fastplong --version
metaspades.py --version
flye --version
metaquast --version
```

```
conda activate meta_annot_env
bakta --version
amrfinder --version
conda activate meta_abricate_env
abricate --version
conda activate meta_env
```

If you used the script, `bakta`, `amrfinder` and `abricate` answer inside `meta_env` as well, and the `conda activate` lines are not needed.

Note the capital letters in `NanoStat` — Linux filenames are case-sensitive, and `nanostat` will not be found. If `metaspades.py` is missing but `spades.py --version` works, that is fine; the metagenomics mode is then `spades.py --meta`. `metaQUAST` arrives inside the `quast` package, so if `metaquast` is not found, try `metaquast.py --version`; whichever answers is the one you use from then on.

Then install the databases. The programs and their databases are separate downloads, and a tool with no database installed will fail at the moment you try to use it rather than now [25, 26, 27]:

```
mkdir -p ~/workshop/databases
conda activate meta_annot_env
bakta_db download --output ~/workshop/databases --type light
amrfinder -u
conda activate meta_abricate_env
abricate --setupdb
abricate --list
conda activate meta_env
```

`bakta_db download` cannot resume: if the 1.3 GB download stops partway, its next attempt starts from zero. The setup script fetches the same file in a way that resumes, so on a slow connection it is worth getting the project folder.

`--type light` is deliberate: Bakta's full database runs to tens of gigabytes, and the light one is the right size for a workshop laptop. Write down the path that `bakta_db download` reports — you pass it to every `bakta` run as `--db`. Record the database versions alongside the tool versions; the same command against a different database version can give a different answer, so the version is part of your result.

Step 5. OPERA-MS — only if your trainer asks for it. The hybrid assembler is not a conda package. It is built from source and carries several legacy dependencies, so it is the one tool here that regularly defeats a laptop, Apple silicon especially [28]:

```
cd ~/workshop
git clone https://github.com/CSB5/OPERA-MS.git
cd OPERA-MS
make
perl OPERA-MS.pl check-dependency
```

If the dependency check reports problems, run `perl tools_opera_ms/install_perl_module.pl` and check again. If it still fails, **stop and use the workshop server**. Do not upgrade the bundled dependencies to make the check pass — OPERA-MS is pinned to the versions it ships with, and a session spent on this is a session not spent on metagenomics.

Step 6. Confirm the whole setup in one go.

```
conda activate meta_env
conda info --envs
fastqc --version && flye --version && bakta --version
```

```
# conda environments:
#
base                        /home/you/miniforge3
meta_abricate_env          /home/you/miniforge3/envs/meta_abricate_env
meta_annot_env             /home/you/miniforge3/envs/meta_annot_env
meta_env                   * /home/you/miniforge3/envs/meta_env
protein_modeling           /home/you/miniforge3/envs/protein_modeling
```

All four workshop environments are listed (`protein_modeling`, `meta_env` and its two helpers), and the `*` is beside the active one. Installed by hand, `bakta --version` answers in `meta_annot_env` rather than here. To go back to the protein session: `conda deactivate`, then `conda activate protein_modeling`.

Checkpoint: `conda info --envs` lists `protein_modeling`, `meta_env`, `meta_annot_env` and `meta_abricate_env`, all ten tools in Step 4 print a version, and you can state where your Bakta database lives.

This module installs the tools; it does not run them. The commands for the pipeline itself — QC, cleaning, assembly, evaluation, annotation and screening, in order — are in section C6 of the [command cheatsheet](#).

Part 5 — Finishing and fixing

Everything you should now have

Module	You can prove it by
1 — WSL (Windows)	<code>wsl --list --verbose</code> shows Ubuntu at version 2
2 — Mac ready	<code>uname -m</code> printed <code>arm64</code> or <code>x86_64</code> , and <code>~/workshop</code> exists
3 — conda	Your prompt shows <code>(protein_modeling)</code> , the version test prints without error, and <code>pymol</code> opens a window
4 — Git	<code>git config --global --get user.name</code> prints your name
5 — GitHub	You can sign in at <code>github.com</code>
6 — VS Code	A local commit exists in <code>git-practice</code> ; publishing is optional
7 — metagenomics (if used)	<code>conda info --envs</code> lists <code>meta_env</code> , and <code>flye --version</code> answers inside it

Before the next session, open a fresh terminal and run `conda activate protein_modeling` once — and `conda activate meta_env` too, if you did Module 7. If they work first time, you are ready.

If something fails

Symptom	Likely cause	What to do
<code>wsl</code> is not recognised in PowerShell	Older Windows build, or PowerShell not run as administrator	Check your build under Settings → System → About [1]
WSL install stalls at 0%	Download route blocked	Try <code>wsl --install --web-download -d Ubuntu</code> [1]
Ubuntu asks for a password and nothing appears	Normal behaviour	Type it anyway and press Enter
<code>conda: command not found</code>	The terminal was opened before conda was installed, or initialisation was skipped	Open a new terminal; then <code>~/miniforge3/bin/conda init bash</code> [4]
conda works in PowerShell but the tools do not	You installed native Windows Miniforge	Use the Ubuntu terminal; the bioconda tools are Linux and macOS only
The Miniforge download is only a few kilobytes	The download failed and saved an error page	Delete the file and download it again. Do not run it
Permission denied, or an urge to use <code>sudo conda</code>	Installing outside your home folder	Work in your home folder; never use <code>sudo</code> with conda
<code>conda create</code> seems frozen	It is resolving and downloading packages	Give it several minutes before worrying
<code>pymol</code> runs but no window appears (Windows)	WSLg is missing or out of date	In PowerShell, <code>wsl --update</code> then <code>wsl --shutdown</code> , and reopen Ubuntu. On an older Windows 10, use the Windows-native PyMOL in Module 3 [2]
Git is missing inside VS Code	Installed in a different system, or VS Code needs a restart	Confirm <code>git --version</code> in the same terminal, then restart VS Code [10, 11]
A metagenomics package “is not available from current channels”	Channels missing, or no build for your CPU	Include both <code>-c conda-forge -c bioconda</code> with <code>--strict-channel-priority</code> ; on Apple silicon, ask about the server [15]
<code>mamba create</code> or <code>install</code> sits on “Solving environment”	A large package set being resolved	Give it several minutes. If it fails, build a fresh environment rather than repairing one [16]

Symptom	Likely cause	What to do
A tool installs but reports no database	Program and database are separate downloads	<code>bakta_db list</code> , <code>amrfinder --update</code> , <code>abricate --list</code> [25, 26, 27]
<code>nanostat</code> : command not found but it is installed	Wrong capitalisation — Linux is case-sensitive	The command is <code>NanoStat</code> [18]
Assembly stops with <code>Killed</code>	The machine ran out of memory	A smaller input, or the workshop server. The command was not wrong
<code>make</code> or the OPERA-MS dependency check fails	Legacy dependencies, common on Apple silicon	Run its dependency check, then use Linux/WSL or the server. Do not upgrade its bundled parts [28]
GitHub sign-in or publish fails	Unverified email, or an incomplete browser prompt	Verify the email and repeat the prompt. Never paste a password or token into a workshop file, and do not force-push [8, 12]

When you ask for help, bring three things: the exact command you typed, the full error message, and the output of `conda info --envs`. That is usually enough to find the problem immediately. Copy the error text before you start trying fixes — the first error is the informative one, and it is easy to lose.

Sources

- [1] Microsoft, WSL installation: <https://learn.microsoft.com/en-us/windows/wsl/install>
- [2] Microsoft, WSL GUI apps: <https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui-apps>
- [3] Microsoft, WSL development environment: <https://learn.microsoft.com/en-us/windows/wsl/setup/environment>
- [4] Miniforge installation: <https://github.com/conda-forge/miniforge/blob/main/README.md>
- [5] Conda, managing environments: <https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html>
- [6] Git downloads: <https://git-scm.com/downloads>
- [7] Git, first-time setup: <https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup>
- [8] GitHub, creating an account: <https://docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github>
- [9] VS Code setup overview: <https://code.visualstudio.com/docs/setup/setup-overview>
- [10] VS Code with WSL: <https://code.visualstudio.com/docs/remote/wsl>
- [11] VS Code source-control quickstart: <https://code.visualstudio.com/docs/sourcecontrol/quickstart>
- [12] VS Code GitHub integration: <https://code.visualstudio.com/docs/sourcecontrol/github>
- [13] Ubuntu, package management: <https://documentation.ubuntu.com/server/how-to/software/package-management/>
- [14] Homebrew: <https://brew.sh>
- [15] Bioconda, using the channel: <https://bioconda.github.io/>
- [16] Mamba documentation: <https://mamba.readthedocs.io/en/latest/>
- [17] FastQC: <https://www.bioinformatics.babraham.ac.uk/projects/fastqc/>
- [18] NanoStat: <https://github.com/wdecoster/nanostat>
- [19] fastp: <https://github.com/OpenGene/fastp>
- [20] fastplong: <https://github.com/OpenGene/fastplong>
- [21] SPAdes and metaSPAdes: <https://ablab.github.io/spades/>
- [22] Flye and metaFlye: <https://github.com/mikolmogorov/Flye>
- [23] metaQUAST: <https://quast.sourceforge.net/metaquast>
- [24] Bandage: <https://rrwick.github.io/Bandage/>

- [25] Bakta: <https://github.com/oschwengers/bakta>
- [26] NCBI AMRFinderPlus: <https://github.com/ncbi/amr/wiki>
- [27] ABRicate: <https://github.com/tseemann/abricate>
- [28] OPERA-MS: <https://github.com/CSB5/OPERA-MS>
- [29] conda-forge PyMOL package: <https://anaconda.org/conda-forge/pymol-open-source>